Step 6 — Metadata

Fields & Commit | Problem Statement | File Paths

The `metadata.json` file already exists in `.helix/`. Open it and fill in the fields:

```
[
  {
    "problem_statement": "",
    "problem_statement_variant": "",
    "hints": "",
    "FAIL_TO_PASS": "",
    "PASS_TO_PASS": ""
  }
]
```

Copy

- problem_statement[1] — Your LLM prompt (see Problem Statement tab for guidance on writing it).
- problem_statement_variant[2] — A more ambiguous restatement of the problem (see Problem Statement tab for guidance).
- hints[3] — Additional context that helps narrow the solution. Include relevant file paths, function names, or error messages.
- FAIL_TO_PASS[4] — Comma-separated list of fail-to-pass test file paths.
- PASS_TO_PASS[5] — Comma-separated list of pass-to-pass test file paths.

1. The only input the agent receives — make it count
2. Same task, described less precisely
3. Never leave this empty
4. Tests that fail before your fix and pass after
5. Tests that should pass both before and after

Commit the metadata file

Commit using the `[meta]` convention:

```
git add .helix/metadata.json
git commit -m "[meta] add metadata.json"
git push origin golden-solution
```

Copy

Commit metadata.json

Make sure `metadata.json` is committed inside `.helix/` alongside your `Dockerfile.helix` and `run-tests-eval.sh`. Your branch should now have three commits: `[sol]`, `[f2p]`, and

`[meta]`.

Variant rubric

The variant is the SAME task described less precisely — a CS student or new contributor's voice, not a senior's.

#	Criterion	What "1" looks like
1	Voice shift	Sounds like a user describing the pain, not the fix. "Is broken" language, not "introduce a Copyable...".
2	No symbol leakage	Does not name the classes/methods/files the original statement names.
3	Same task	The same F2P + P2P set must pass against both statements.
4	Length	50–70% of the original. Same-length variants are usually synonym swaps, not real variants.

For worked references, see the variant pairs (TypeORM and SUM examples) on the Problem Statement tab.

Hints rubric

Hints exist to keep an agent from burning steps searching the wrong area. They are SIGNPOSTS, not blueprints.

#	Criterion	What "1" looks like
1	Locations	Names at least one file path or module where the change lives.
2	Symbols	Names at least one function/class that's involved.
3	No implementation	Does NOT describe the fix. "Look at the loop in `merge_configs` " is fine; describing the fix is not.
4	Diagnostic signal	For bug-fix tasks, includes the error message or symptom a user would see.

Good

Signpost

The merge logic lives in `config/merger.py` in `merge_configs()`. Existing tests are in `tests/test_merger.py`. Users hit this when calling `--config base.yaml --config override.yaml` with nested sections.

Bad

Leaks the solution

In `config/merger.py`, replace the `result.update(override)` call with a recursive function that checks if both values are dicts and merges them.

Previous

Next